

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第6章 指针

- ◆ 6.1 指针定义与使用
- ◆ 6.2 指针与函数
- ◆ 6.3 指针与数组
- ◆ 6.4 指针与字符串
- ◆ 6.5 指针数组与多级指针
- ◆ 6.6 指针与动态内存分配



6.1 指针定义与使用

- 指针类型是C语言的一种特殊数据类型。正确而灵活地应用指针，可以有效地表示复杂的数据类型、有效地处理字符串和数组，动态地分配内存、处理内存地址，实现主调函数和被调函数之间共享变量等。
- C语言中的指针是一种数据类型，指针类型和int，float等数据类型没什么大区别，需要特别关注的是指针这种数据类型所存储的数据的属性。
- 学习指针应该充分地理解变量名、变量的值、变量的地址这三个概念的含义，理解三者之间的关系。

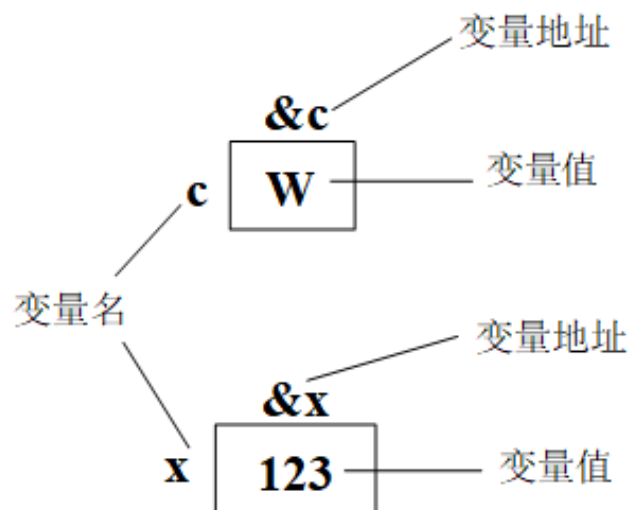
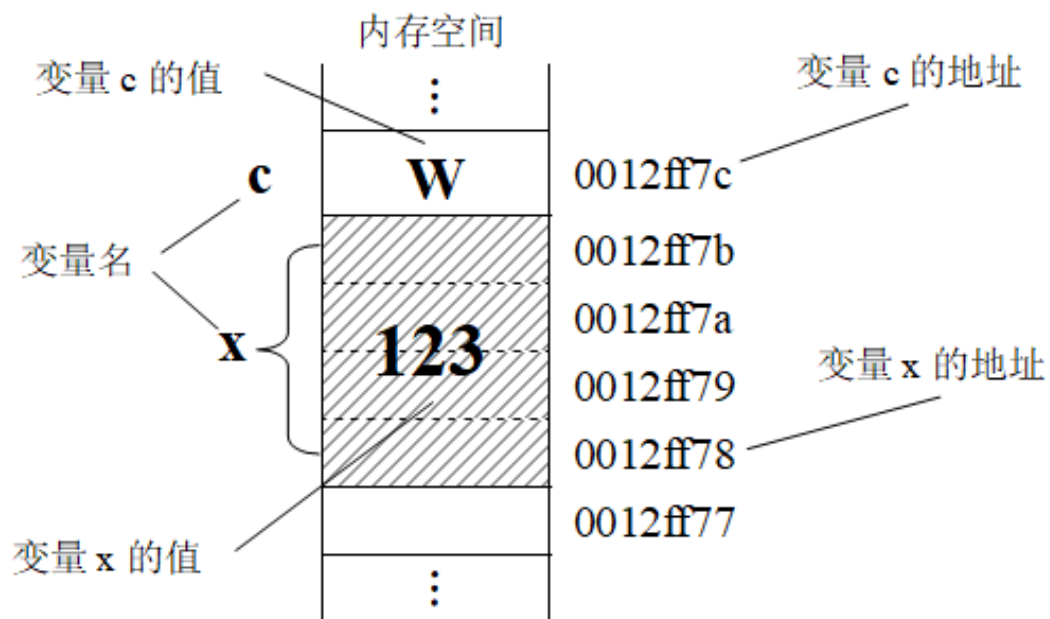


6.1.1 指针的引出

1. 变量在内存中的存储

计算机内存是由一片连续的存储单元组成，操作系统给每个内存单元一个编号，这个编号称为内存单元的地址。

char c='W'; 存取数据时，变量的地址是第一个存储单元(字节)的地址，系统根据变量的类型确定存取的字节数。
int x=123;



6.1.1 指针的引出

2. 指针的引出

【问题描述6.1】利用函数实现三个数按从小到大的顺序输出。

分析：3个数排序最多需要进行3次比较和交换

1, 2, 3	2, 1, 3	3, 1, 2	3, 2, 1
0次交换	1次交换	2次交换	3次交换

解题方法：

编写swap函数实现数据的交换，通过在main函数中3次调用swap函数来实现3个数排序



6.1.1 指针的引出

【问题描述6.1】参考程序（未使用指针变量）

```
#include <stdio.h>
```

```
void swap(int a, int b)
{   int temp;
    temp=a; a=b; b=temp;
}
```

```
int main( )
{   int x1, x2, x3;
    scanf("%d%d%d", &x1, &x2, &x3);
    if (x1>x2) swap(x1,x2);
    if (x1>x3) swap(x1,x3);
    if (x2>x3) swap(x2,x3);
    printf("%d,%d,%d", x1, x2, x3);
    return 0;
}
```

◆ 程序存在的问题

程序运行后，数据并没有排序，因为swap函数中的参数a和b与main函数中的变量x1, x2, x3存储在不同的内存空间。swap函数中a、b的交换并不能使main函数中的x1, x2, x3进行交换。

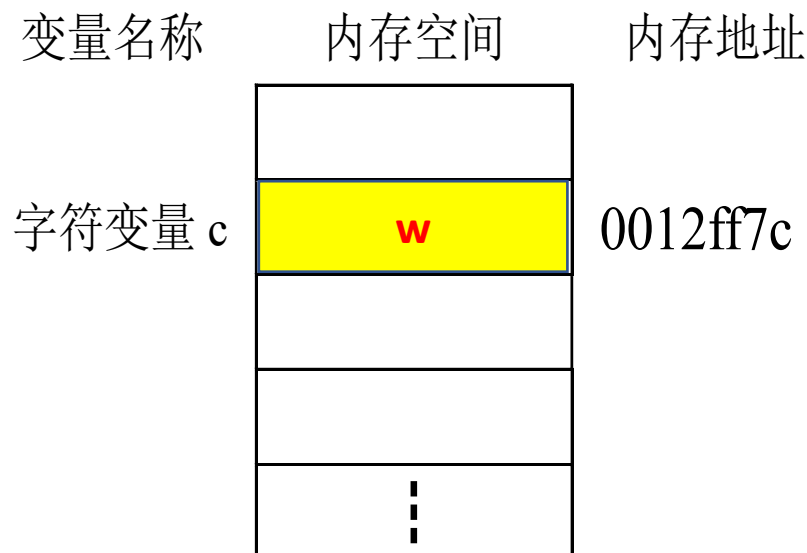
◆ 解决方法：如果main函数传递给swap函数的不是数据本身，而是存放数据的地址，那么就可以对main中的数据进行交换了。

6.1.1 指针的引出

C语言系统定义了称为**指针**的数据类型，专门用于**存放**其他数据类型的**变量的地址**。

- 变量的“**直接访问**”：
通过变量名访问该内存单元值的方法。

```
char c;  
c='W';  
c=c+32  
putchar( c );
```



6.1.1 指针的引出

- 变量的地址就是变量的指针
- 指针变量: 存放其他变量或函数的地址的变量。
 - 变量的“间接访问”: 通过指针变量访问某内存单元值的方法。

将字符变量 **c** 的地址存放在指针变量 **p** 中, 访问 **c** 时, 需要先找到指针变量 **p**, 再由 **p** 中存放的地址找到 **c**

变量名称	内存空间	内存地址
字符变量 c	W	0012ff7c
	⋮	
指针变量 p	0012ff7c	1012af80
	⋮	

图 6.3 指针变量与变量地址的关系

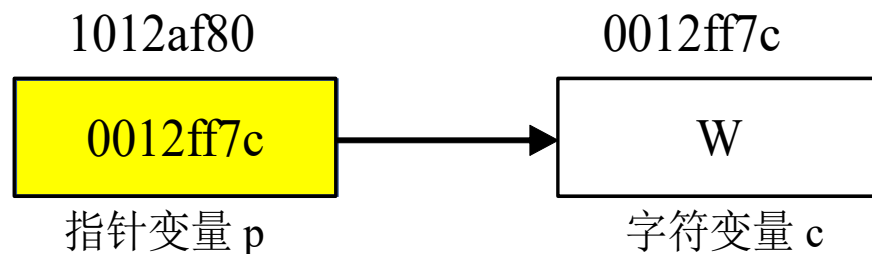


图 6.4 指针变量间接引用普通变量示意图

6.1.2 指针变量的定义

◆ 指针变量的定义形式: 类型标识符 * 指针变量名;

说明:

- ① *表示这是定义一个指针变量。
- ② 变量名即为定义的指针变量名。
- ③ 类型标识符表示本指针变量所指向的变量的数据类型, 也称为指针变量的基类型。

例: `int *p1;` // p1是一个指向整型变量的指针变量

`double *p2;`

`float *p3;`

`char *p4;`

☹ 注意:

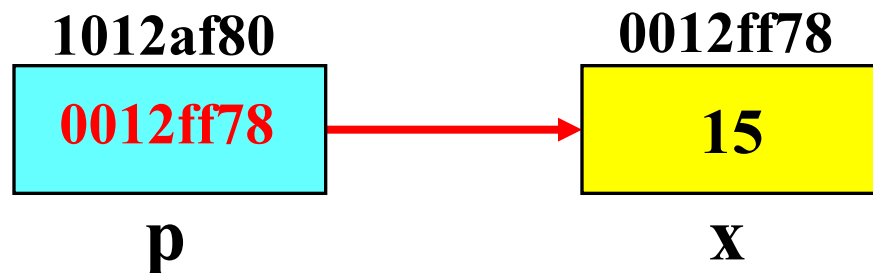
一个指针变量只能指向同类型的变量。

6.1.3 指针变量的使用

1. 指针变量的赋值

(1) 通过地址运算 “&”赋值

例: `int x=15, *p;`
`p=&x;`



(2) 指针变量的初始化

定义一个指针变量时给指针变量赋初值

例: `int x=15, *p=&x;`

6.1.3 指针变量的使用

1. 指针变量的赋值

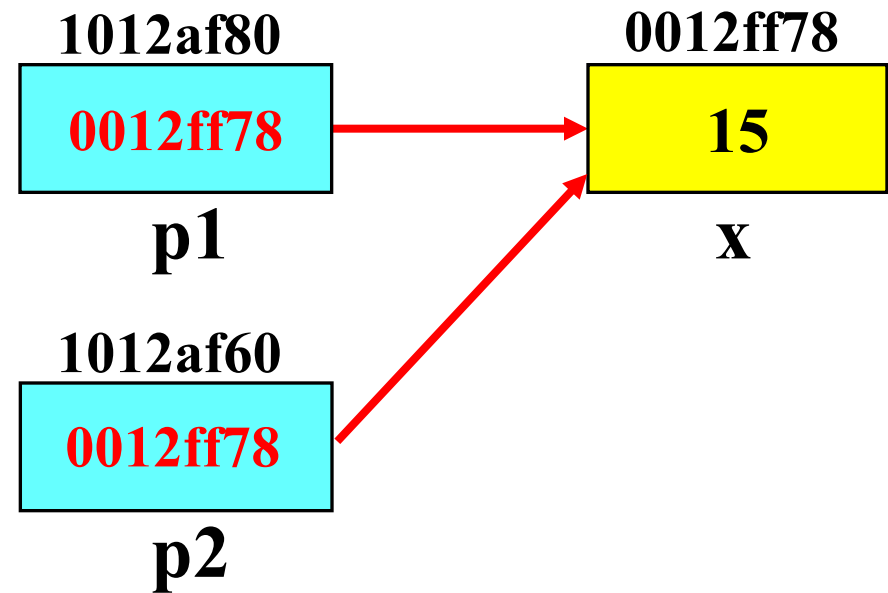
(3) 通过其它指针变量赋值

把一个指针变量的值赋给另一个指针变量，这样两个指针变量均指向同一个地址

例：int x=15, *p1 = &x, *p2;

p2=p1;

注意：当把一个指针变量的地址值赋给另一个指针变量时，赋值号两边指针变量所指的数据类型必须相同。



int x, *pi=&x;

float *pf;

~~pf=pi;~~



将整型指针变量的地址值赋给指向float型的指针变量是错误的

6.1.3 指针变量的使用

1. 指针变量的赋值

(4) 用NULL给指针变量赋空值

例: `int x=15, *p=NULL;`
`p=NULL;`

NULL是在stdio.h头文件中定义的预定义标识符, 在C语言中当指针值为NULL时, 指针不指向任何有效数据, 因此在程序中为了防止错误地使用指针来存取数据, 常在指针未使用之前先赋初值为NULL。

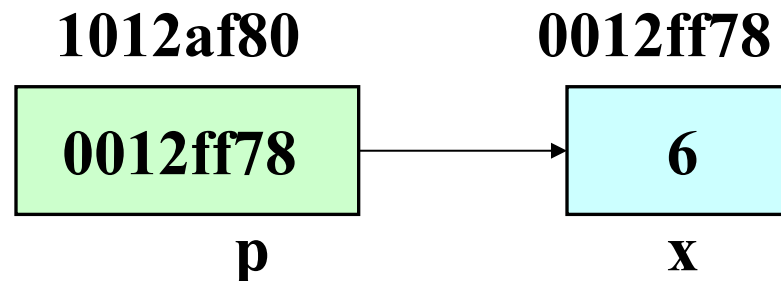
注意: NULL可以赋值给指向任何类型的指针变量。



6.1.3 指针变量的使用

2. 指针运算符

指针运算符 “*****” 是单目运算符，运算对象只能是指针变量或地址，可以用指针运算符来存取相应的存储单元中的数据。



例: **int x, *p;** //定义时, * 号表示该变量是指针变量

p=&x; //对指针变量赋值, 令p指向变量x

***p=6;**

//使用时, *p表示该指针变量所指向的变量
即: ***p=6;** 与 **x=6;** 等价

scanf("%d", p); 和 **scanf("%d", &x);** 等价

注意: 指针运算与取地址运算互为逆运算

int a, *p;

p = &a;

&*p → &(*p) → &a → p

***&a → *(&a) → *p → a**

6.1.3 指针变量的使用

【例6.1】由键盘输入一个正整数，求出其最高位数字。

分析：若输入258，则 $258/100=2$ ；若输入3486，则 $3486/1000=3$ ；由于输入的正整数可以是任意位数，每次整除的数不一样，所以采用另一种方法， $258/10=25$ ， $25/10=2$ ，即对输入的正整数整除10，当结果大于等于10时重复进行整除10的操作。

```
#include<stdio.h>
```

```
int main( )
```

```
{ int i, *p;
```

```
    p=&i;           //指针变量指向变量i
```

```
    printf("请输入一个正整数: ");
```

```
    scanf("%d", p); //等价于scanf("%d", &i);
```

```
    while (*p >=10) //用循环求出该正整数的最高位数字
```

```
    { *p= *p/10; }
```

```
    printf("最高位数字是: %d\n", *p);
```

```
    return 0;
```

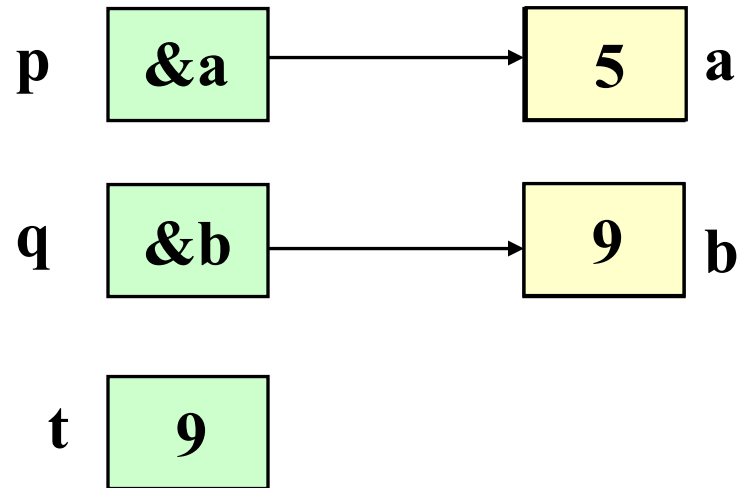
```
}
```

6.1.3 指针变量的使用

【例6.2】输入两个整数，按照由小到大的顺序输出。

参考程序1

```
#include<stdio.h>
int main()
{ int a, b, *p, *q, t;
  scanf("%d%d", &a, &b);
  p=&a; q=&b;
  if(*p>*q)
  { t=*p;
    *p=*q;
    *q=t;
  }
  printf("%d, %d\n", a, b);
  return 0;
}
```



6.1.3 指针变量的使用

【例6.2】输入两个整数，按照由小到大的顺序输出。

参考程序2

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b, *p, *q, *t;
```

```
scanf("%d%d", &a, &b);
```

```
p=&a; q=&b;
```

```
if(*p>*q)
```

```
{ t=p;
```

```
  p=q;
```

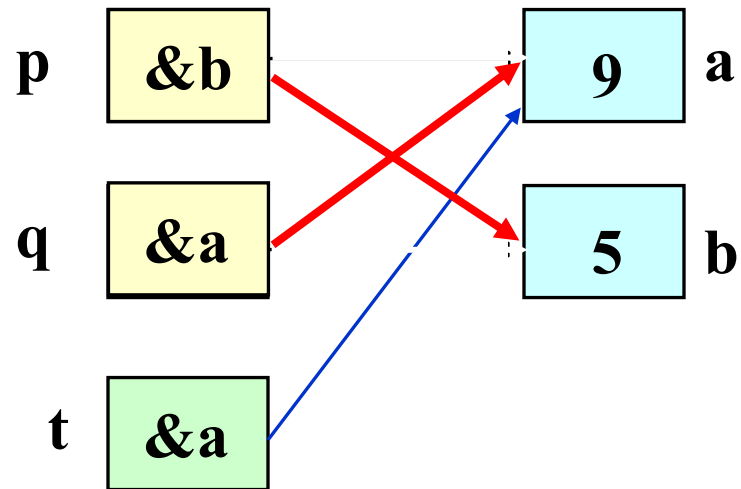
```
  q=t;
```

```
}
```

```
printf("%d, %d\n", a, b);
```

```
return 0;
```

```
}
```



```
printf("%d, %d\n", *p, *q);
```

6.2 指针与函数

6.2.1 指针作为函数参数

【例6.3】编写一个函数实现main函数中两个变量的值的交换。

```
#include <stdio.h>
```

```
void swap( int *p, int *q);
```

```
int main( )
```

```
{ int x1, x2;
```

```
scanf("%d%d", &x1, &x2);
```

```
swap(&x1, &x2);
```

```
printf("x1=%d, x2=%d\n", x1, x2);
```

```
return 0;
```

```
}
```

```
void swap( int *p, int *q)
```

```
{ int temp;
```

```
temp=*p; *p=*q; *q=temp;
```

```
}
```

main

x1

9

x2

5

说明: 该方法是交换p和q所指向的变量的值, 即交换main函数中x1和x2的值, 所以输出为: x1=9, x2=5

6.2.1 指针作为函数参数

【问题描述6.1】 参考程序（使用指针变量）

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp;
    temp=*a;  *a=*b;  *b=temp;
}

int main( )
{
    int x1, x2, x3;
    scanf("%d%d%d", &x1, &x2, &x3);
    if (x1>x2) swap(&x1, &x2);
    if (x1>x3) swap(&x1, &x3);
    if (x2>x3) swap(&x2, &x3);
    printf("%d,%d,%d", x1, x2, x3);
    return 0;
}
```

//不用指针变量作参数，无法实现
//main函数中3个变量的排序

```
#include <stdio.h>
void swap(int a, int b)
{
    int temp;
    temp=a; a=b; b=temp;
}

int main( )
{
    int x1, x2, x3;
    scanf("%d%d%d", &x1, &x2, &x3);
    if (x1>x2) swap(x1,x2);
    if (x1>x3) swap(x1,x3);
    if (x2>x3) swap(x2,x3);
    printf("%d,%d,%d", x1, x2, x3);
    return 0;
}
```



6.2.1 指针作为函数参数

【例6.4】编写函数实现对main函数传来的两个浮点数求其和值与差值，并存储到主函数的相应变量中。

```
#include<stdio.h>
```

```
void fun(float a, float b, float *c, float *d)  main
```

```
{  *c=a+b;
```

x 1.5

```
    *d=a-b;
```

```
}
```

y 0.6

```
int main()
```

```
{  float x, y, sum, diff;
```

```
    printf("请输入两个浮点数: ");
```

sum 2.1

```
    scanf("%f%f", &x, &y);
```

```
    fun(x, y, &sum, &diff );
```

diff 0.9

```
    printf("两数之和=%.2f\n两数之差=%.2f\n", sum, diff );
```

```
    return 0;
```

```
}
```

6.2.2 函数返回指针

- 函数的返回值可以是一个指针类型的数据(即地址)

返回指针的函数定义格式:

类型标识符 *函数名(形参列表)
{ 函数体语句; }

说明: 在函数名前加 * , 表明这是一个指针型函数,
函数的返回值是一个指针(即函数返回一个地址)

例: int * fun (int a, int b)
{ 函数体 ; }

6.2.2 函数返回指针

【例6.6】 利用指针函数求两个数中的最大值。

```
#include <stdio.h>
```

```
int *maxp(int *, int *);
```

```
int main( )
```

```
{  int x, y, *p;
```

```
    scanf("%d%d", &x, &y);
```

```
    p=maxp(&x, &y);
```

```
    printf("\nmaxp=%d", *p);
```

```
    return 0;
```

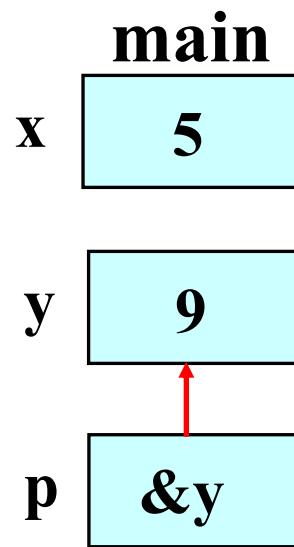
```
}
```

```
int *maxp(int *a, int *b)
```

```
{
```

```
    return *a>*b? a:b;
```

```
}
```



6.3 指针与数组

6.3.1 一维数组与指针

1. 一维数组及元素的地址表示

`int a[5] = { 1, 2, 3, 4, 5 };`

数组的首地址用数组名表示

元 素		地 址	
*a	a[0]	&a[0]	a
*(a+1)	a[1]	&a[1]	a+1
*(a+2)	a[2]	&a[2]	a+2
*(a+3)	a[3]	&a[3]	a+3
*(a+4)	a[4]	&a[4]	a+4

6.3.1 一维数组与指针

2. 定义一个指向一维数组元素的指针变量

```
int a[5] = { 1, 2, 3, 4, 5 };
```

```
int *p;
```

```
p=&a[0]; //把元素a[0]的地址赋给指针变量p
```

```
p=a; //数组名代表数组的首地址， p指向元素a[0]
```

```
int *p=&a[0];
```

```
int *p=a;
```

对指针变量进行初始化

6.3.1 一维数组与指针

3. 通过指针引用数组元素

例: `int *p, a[5] = { 1, 2, 3, 4, 5 };`

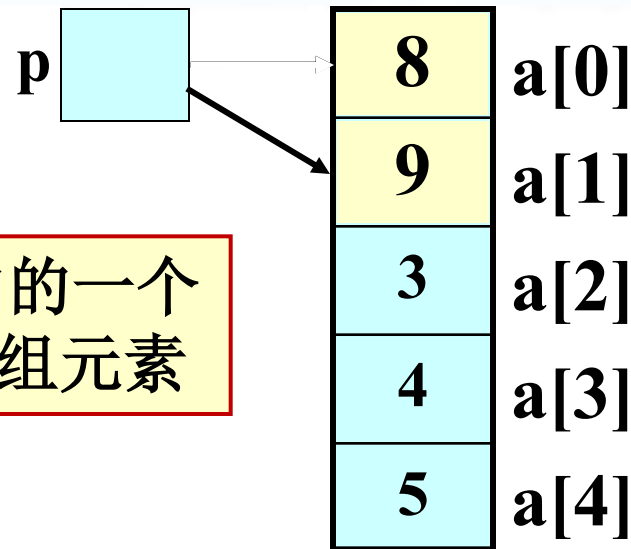
`p=a;`

`*p=8;`

`p=p+1;`

`*p=9;`

C语言规定若p指向数组中的一个元素,则p+1指向下一个数组元素



若`p=a`, 或`p=&a[0]`;

则`p+i`和`a+i`是元素`a[i]`的地址, 而`*(p+i)`和`*(a+i)`就是数组元素`a[i]`

数组元素的访问方法 { 下标法: 数组名[下标]
指针法: `*(数组名+下标)` 或 `*指针变量`

```
for(i=0; i<5; i++)  
    printf("%d", *(a+i));
```

```
for(p=a; p<(a+5); p++)  
    printf("%d", *p);
```

6.3.1 一维数组与指针

3. 通过指针引用数组元素

例： `int *p, a[5] = { 1, 2, 3, 4, 5 };`

`p=a;`

	下标法	地址法	指针法
第k个元素	<code>a[k]</code>	<code>*(a+k)</code>	<code>*(p+k)</code>
第k个元素的地址	<code>&a[k]</code>	<code>a+k</code>	<code>p+k</code>

注意：

- ①指针变量也可以加下标 `p[k]` 等价于 `a[k]`
- ②用指针变量访问数组元素时要注意下标是否越界

6.3.1 一维数组与指针

例: 将数组a中全部元素加1, 再输出a

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[5] = {1, 3, 5, 7, 9}, *p, j;
```

```
  for ( p=a; p<a+5; p++ )
```

```
    printf("%3d", *p);
```

```
  printf("\n");
```

```
  for ( j=0; j<5; j++ )
```

```
    a[j]=a[j]+1;
```

```
  p=a;
```

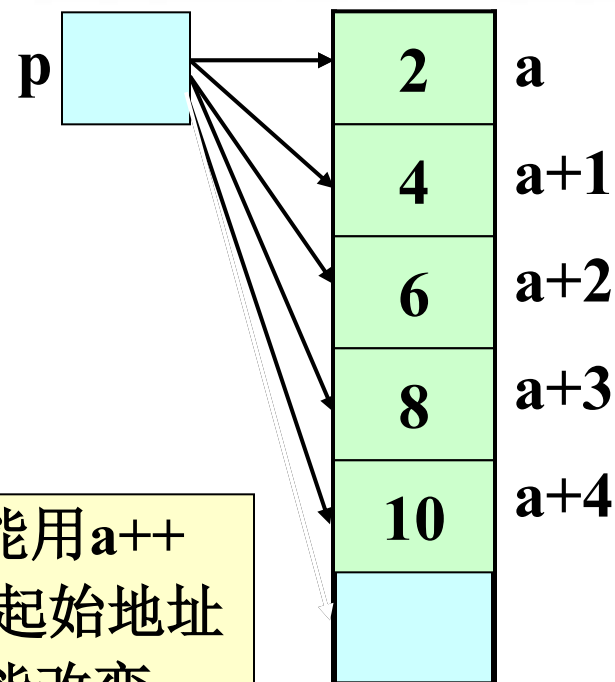
```
  for ( j=0; j<5; j++ )
```

```
    printf("%3d", *(p+j));
```

```
  printf("\n");
```

```
  return 0;
```

```
}
```



可以用`p++`, 但不能用`a++`
因为`a` 代表数组的起始地址
它是地址常量, 不能改变,
而`p` 是一个指针变量

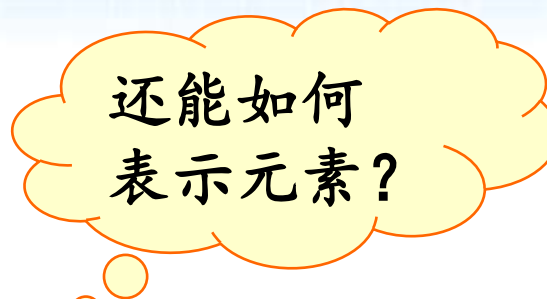
使用指针变量要
注意它的当前值

可以换成`*(a+j)`吗?

6.3.1 一维数组与指针

```
int a[5] = { 1, 2, 3, 4, 5 }, *p;
```

```
p=a;
```



元素	地址	元素
a[0]	a	*a
a[1]	a+1	*(a+1)
a[2]	a+2	*(a+2)
a[3]	a+3	*(a+3)
a[4]	a+4	*(a+4)

地址	元素	元素
p	*p	p[0]
p+1	*(p+1)	p[1]
p+2	*(p+2)	p[2]
p+3	*(p+3)	p[3]
p+4	*(p+4)	p[4]

6.3.2 数组名作为函数参数

- 实参数组名代表该数组首元素的地址，是一个固定的地址，是指针常量。
- 形参作为一个指针变量，是用来接收从实参传递过来的数组首元素地址的。
- 在函数调用开始时，形参的值等于实参数组首元素的地址，但在函数执行过程中，它可以再次被赋值。
- 实际上，C编译器是将形参数组名作为指针变量来处理的。

6.3.2 数组名作为函数参数

(1) 数组名作函数参数

函数定义:

```
void f(int b[])  
{ ...  
}
```

函数调用:

```
int main()  
{  
    int a[10];  
    f(a);  
}
```

(2) 指向数组元素的指针作函数参数

函数定义:

```
void f(int *b)  
{ ...  
}
```

函数调用:

```
int main()  
{  
    int a[10];  
    int *p;  
    p=a;  
    f(p);  
}
```

等价于



等价于

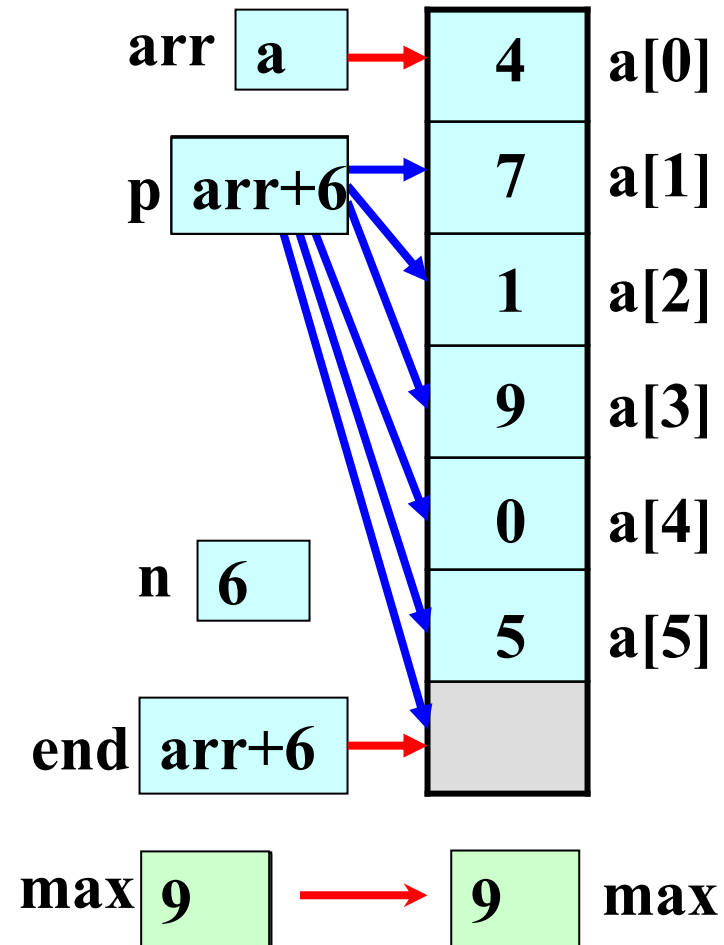


6.3.2 数组名作为函数参数

例6.9 求数组中最大元素(使用指针作函数参数)

```
int fun(int *arr, int n)
{
    int *p, *end, max;
    end=arr+n;  max=*arr;
    for(p=arr+1 ; p<end ; p++)
    {
        if(*p>max)
            max=*p;
    }
    return max;
}

int main( )
{
    int i, a[6], max;
    for(i=0 ; i<6 ; i++)
        scanf("%d",&a[i] );
    max = fun(a, 6);
    printf("max=%d\n", max);
}
```



6.3.2 数组名作为函数参数

【例6.10】编程实现数组元素的逆置。

◆ 参考程序1-- 实参和形参都用数组名

```
#include <stdio.h>
```

```
void reverse1(int x[ ], int n)
```

```
{ int temp, i, j, m=(n-1)/2;
```

```
  for( i=0 ; i<=m ; i++)
```

```
  { j=n-1-i;    temp=x[i];
```

```
    x[i]=x[j];  x[j]=temp;
```

```
  }
```

```
}
```

```
int main( )
```

```
{ int i, a[6]={ 1, 3, 4, 6, 7, 9 };
```

```
  reverse1 (a, 6 );
```

```
  for( i=0; i<6; i++ ) printf("%3d", a[i] );
```

```
  printf("\n");
```

```
  return 0;
```

```
}
```

reverse1

main

x[0]	9	a[0]
x[1]	7	a[1]
x[2]	6	a[2]
x[3]	4	a[3]
x[4]	3	a[4]
x[5]	1	a[5]

6.3.2 数组名作为函数参数

◆ 参考程序2-- 实参用数组名, 形参用指针变量

```
#include <stdio.h>
```

```
void reverse2(int *x, int n)
```

```
{ int temp, m=(n-1)/2;
```

```
  int *p, *i, *j ;
```

```
  j=x+n-1;  p=x+m;
```

```
  for( i=x; i<=p; i++, j-- )
```

```
  { temp=*i; *i=*j; *j=temp; }
```

```
}
```

```
int main( )
```

```
{ int i, a[6]={ 1, 3, 4, 6, 7, 9 };
```

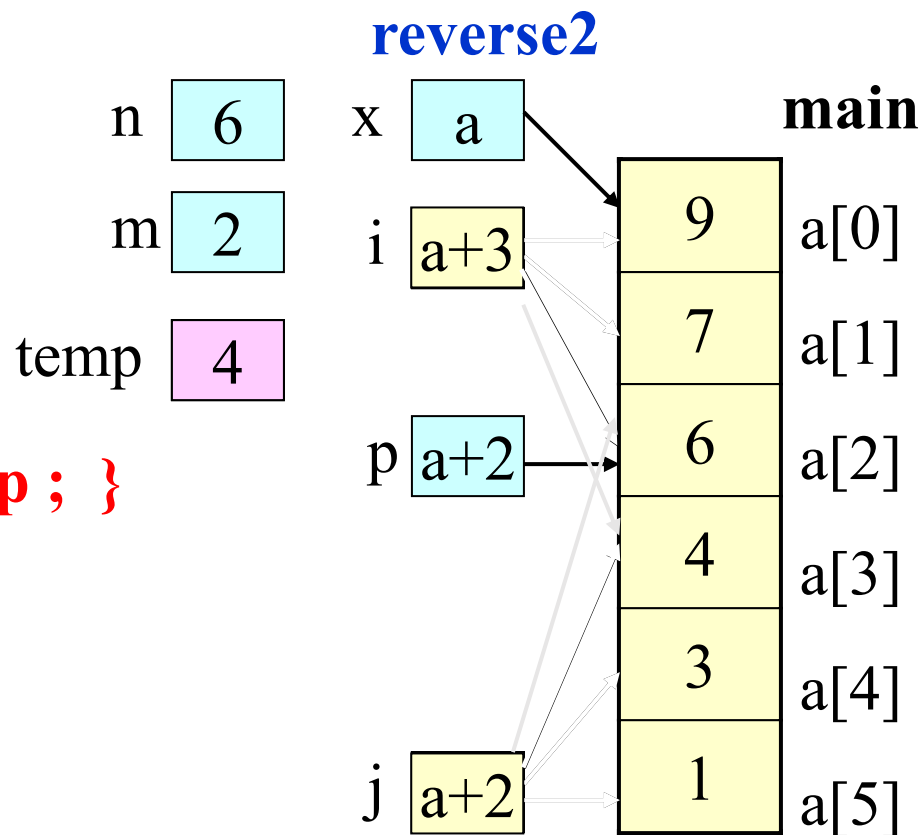
```
  reverse2 (a, 6 );
```

```
  for( i=0; i<6; i++ ) printf("%3d", a[i] );
```

```
  printf("\n");
```

```
  return 0;
```

```
}
```



6.3.2 数组名作为函数参数

◆ 参考程序3-- 实参和形参都用指针变量

```
#include <stdio.h>
```

```
void reverse3(int *x, int n)
```

```
{ int temp, m=(n-1)/2;
```

```
  int *p, *i, *j ;
```

```
  j=x+n-1;   p=x+m;
```

```
  for( i=x; i<=p; i++, j-- )
```

```
  { temp=*i ; *i=*j ; *j=temp ; }
```

```
}
```

```
int main( )
```

```
{ int *p , a[6]={ 1, 3, 4, 6, 7, 9 };
```

```
  p=a;
```

```
  reverse3 (p, 6 );
```

```
  for( p=a ; p<a+6 ; p++ ) printf("%3d", *p );
```

```
  printf("\n");
```

```
  return 0;
```

```
}
```

6.3.2 数组名作为函数参数

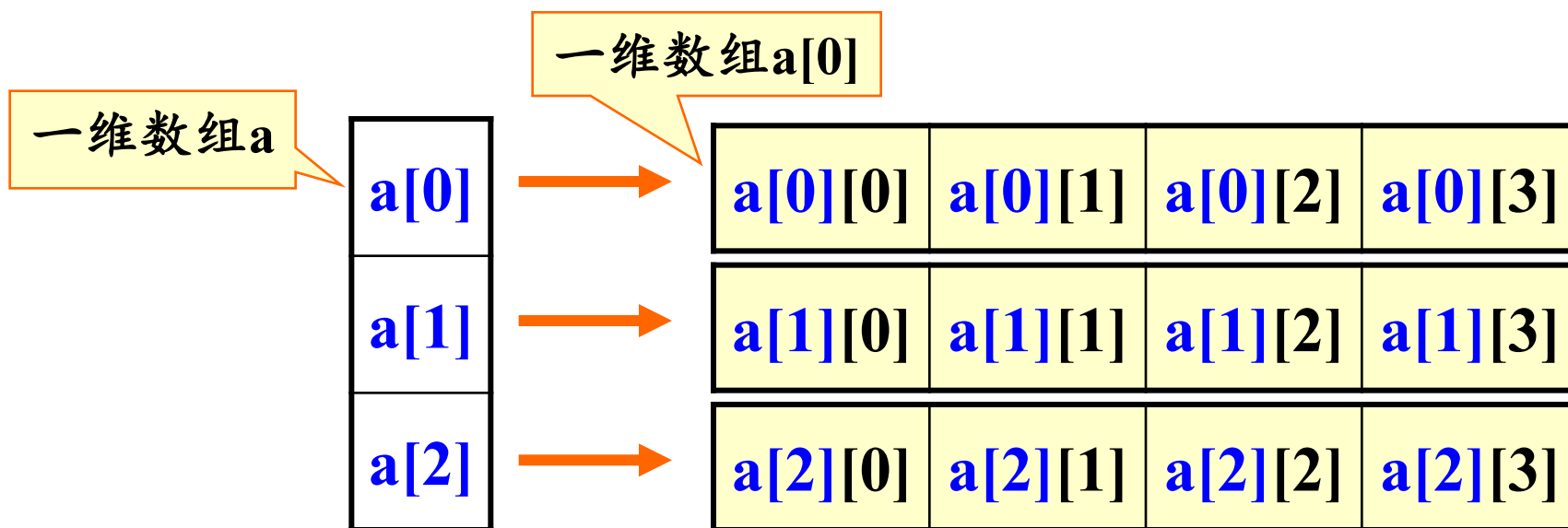
小结: 如果有一个实参数组, 想在函数中改变此数组的元素的值, 实参与形参的对应关系有四种

	实参	形参
1	数组名	数组名
2	数组名	指针变量
3	指针变量	数组名
4	指针变量	指针变量

6.3.3 二维数组与指针

```
int a[3][4];
```

把二维数组a理解成一个一维数组, 它有3个元素: $a[0]$, $a[1]$, $a[2]$



每一个元素又是一个一维数组, 包含4个元素

如 $a[0]$ 是一维数组, 它有4个元素: $a[0][0]$, $a[0][1]$, $a[0][2]$, $a[0][3]$

PS: $a[0]$, $a[1]$, $a[2]$ 并不真的存在

6.3.3 二维数组与指针

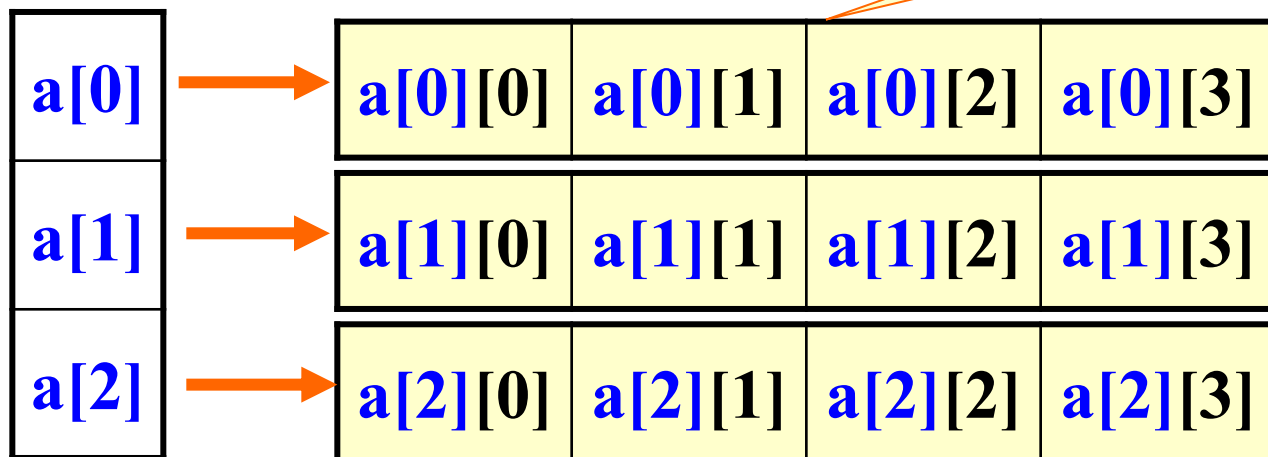
二维数组是由一维数组组成。

列地址

一维数组a[0]

一维数组a

行地址



二维数组必然存在两种截然不同的地址：

一种是指向数组的**行**(指向数组a的地址)，即**行地址**。

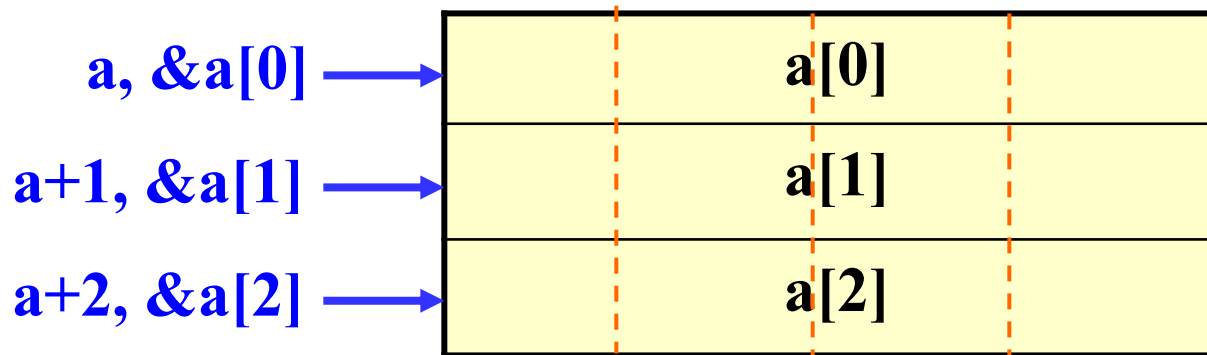
一种是指向数组的**列**(指向数组a[0]的地址)，即**列地址**。

➤ 二维数组的地址——行地址

- 二维数组名 a 表示 **二维数组的首地址**, 也是 **第0行** 的首地址, $a \Leftrightarrow \&a[0]$

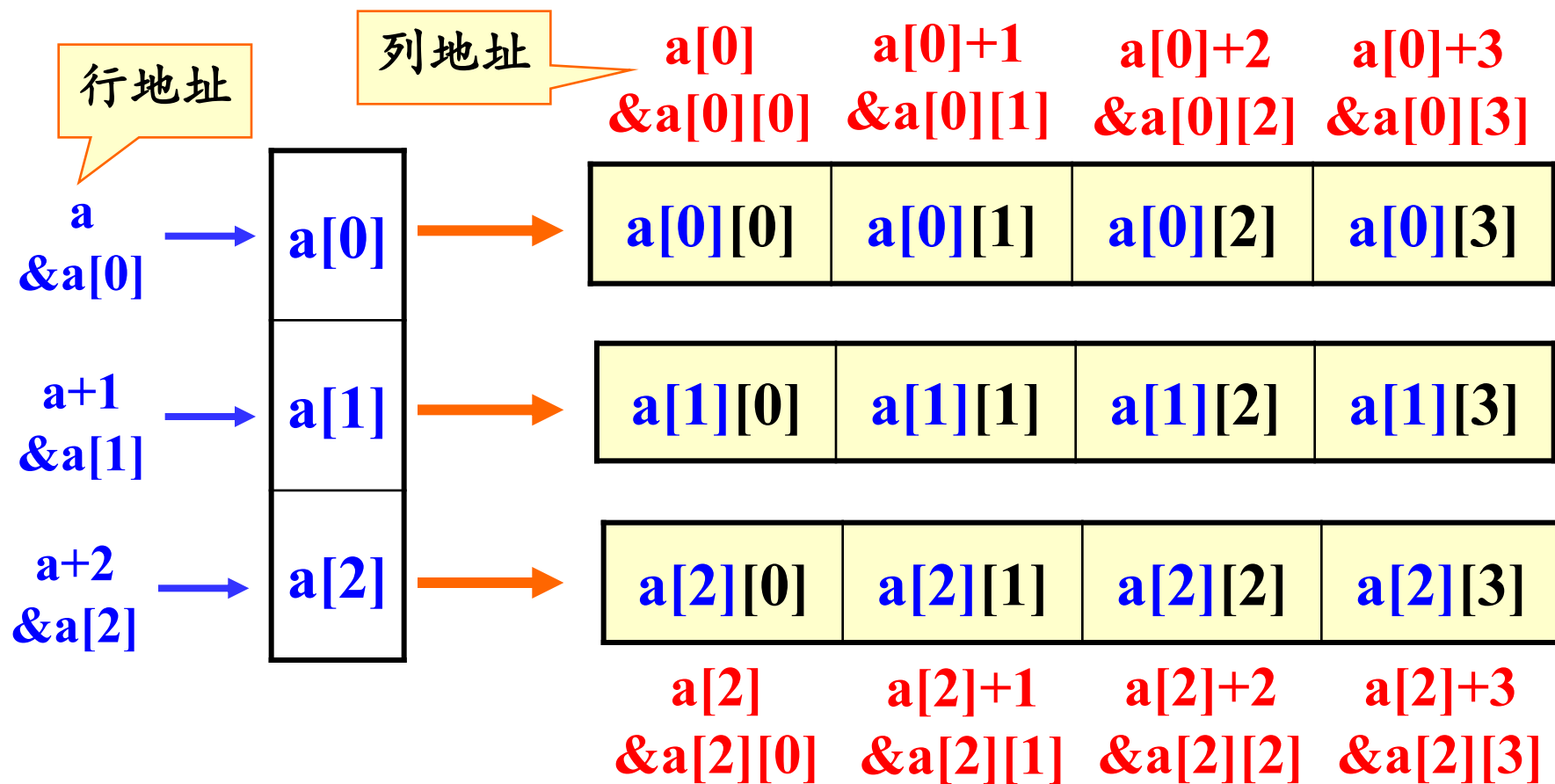
$a+1$ 代表 **第1行** 的首地址, $a+1 \Leftrightarrow \&a[1]$

$a+2$ 代表 **第2行** 的首地址, $a+2 \Leftrightarrow \&a[2]$



➤ 二维数组的地址——列地址

- $a[0]$, $a[1]$, $a[2]$ 是一维数组名, 它们就代表了一维数组的首地址, 因此 $a[0]$ 代表了第0行第0列元素的地址, 即 $a[0] \Leftrightarrow \&a[0][0]$,



➤ 行地址列地址的转换

- 行地址->列地址

方法：加*号

如 $a \rightarrow *a$, $(a+1) \rightarrow *(a+1)$

- 列地址->行地址

- 方法：去*号，或取地址(&)

如 $*a \rightarrow a$, $a[0] \rightarrow \&a[0]$

$a[0] \rightarrow \&a[0]$

$\Leftrightarrow a[0] \rightarrow \&(*a)$

$\Leftrightarrow a[0] \rightarrow a$



➤ 二维数组的地址

思考: $*a, *a+1, *(a+1), *(a+2)+3$ 表示什么?

因 $a \Leftrightarrow \&a[0]$, 所以 $*a \Leftrightarrow *(\&a[0]) \rightarrow a[0] \Leftrightarrow \&a[0][0]$

即: $*a \Leftrightarrow a[0] \Leftrightarrow \&a[0][0]$

$*a$ 表示元素 $a[0][0]$ 的地址

$*a+1$ 表示元素 $a[0][1]$ 的地址

$*(a+1)$ 表示元素 $a[1][0]$ 的地址

$*(a+2)+3$ 表示元素 $a[2][3]$ 的地址

$a, \&a[0] \rightarrow$

$a+1, \&a[1] \rightarrow$

$a+2, \&a[2] \rightarrow$

$*a$ $a[0]$ $\&a[0][0]$	$*a+1$ $a[0]+1$ $\&a[0][1]$		
$a[0][0]$	$a[0][1]$	$a[0][2]$	$a[0][3]$
$a[1][0]$	$a[1][1]$	$a[1][2]$	$a[1][3]$
$a[2][0]$	$a[2][1]$	$a[2][2]$	$a[2][3]$

$*(a+1)$
 $a[1]$
 $\&a[1][0]$

$a[2]+3$
 $\&a[2][3]$
 $*(a+2)+3$

思考

```
int a[3][3] = {1,2,3,4,5,6,7,8,9};
```

1. a 和 $*a$ 的值是否相同? 相同
2. $a+1$ 和 $*(a+1)$ 的值是否相同? 相同
3. $(a+1)+1$ 和 $*(a+1)+1$ 的值是否相同?

$(a+1)+1 = a+2$

第2行的首地址, 即 $\&a[2]$

$*(a+1)+1 = a[1]+1$

第1行第1列的首地址, 即 $\&a[1][1]$

$a \rightarrow 1010$

1014

1018

$a+1 \rightarrow 1022$

$*(a+1)+1 \rightarrow 1026$

1030

$a+2 \rightarrow 1034$

1038

1042

1	$a[0][0]$
2	$a[0][1]$
3	$a[0][2]$
4	$a[1][0]$
5	$a[1][1]$
6	$a[1][2]$
7	$a[2][0]$
8	$a[2][1]$
9	$a[2][2]$

为什么同样是+1,
结果却不同?

➤ 小结

- 二维数组名 **a** 代表二维数组的首地址,

即: 第0行的地址为 $\mathbf{a \Leftrightarrow \&a[0]}$

第1行的地址为 $\mathbf{a+1 \Leftrightarrow \&a[1]}$

第2行的地址为 $\mathbf{a+2 \Leftrightarrow \&a[2]}$

- 二维数组的列地址即元素的地址

元素	地址	元素	地址	元素	地址
$\mathbf{a[0][0]}$	$\mathbf{\&a[0][0]}$	$\mathbf{*a[0]}$	$\mathbf{a[0]}$	$\mathbf{**a}$	$\mathbf{*a}$
$\mathbf{a[0][1]}$	$\mathbf{\&a[0][1]}$	$\mathbf{*(a[0]+1)}$	$\mathbf{a[0]+1}$	$\mathbf{*(*a+1)}$	$\mathbf{*a+1}$
$\mathbf{a[1][0]}$	$\mathbf{\&a[1][0]}$	$\mathbf{*a[1]}$	$\mathbf{a[1]}$	$\mathbf{*(*(a+1))}$	$\mathbf{*(a+1)}$
$\mathbf{a[1][2]}$	$\mathbf{\&a[1][2]}$	$\mathbf{*(a[1]+2)}$	$\mathbf{a[1]+2}$	$\mathbf{*(*(a+1)+2)}$	$\mathbf{*(a+1)+2}$
$\mathbf{a[2][3]}$	$\mathbf{\&a[2][3]}$	$\mathbf{*(a[2]+3)}$	$\mathbf{a[2]+3}$	$\mathbf{*(*(a+2)+3)}$	$\mathbf{*(a+2)+3}$

6.3.3 二维数组与指针

1、指向数组元素的指针变量

例6.13 用指针变量输出二维数组中的元素

```
#include<stdio.h>
```

```
int main( )
```

```
{
```

```
int a[3][4]={1,3,5,7},{2,4,6,8},{9,10,11,12}};
```

```
int *p;
```

```
for( p=a[0]; p<a[0]+12; p++)
```

```
{
```

```
printf("%4d", *p);
```

```
if ( (p-a[0])%4==3 )
```

```
printf("\n");
```

```
}
```

```
}
```

a+12? 错

*a+12? 对

思考:

p=a; 错

p=&a[0][0]; 对

p=*a; 对

p=*a[0]; 错



6.3.3 二维数组与指针

2、行指针变量 (指向一个一维数组的指针变量)

定义格式: 类型名 (***指针变量名**)[数组长度];

如 `int (*p)[4];`

p指向一个包含有4个整型数据的一维数组

用法: `int a[3][4]`
`int (*p)[4];`
`p=a;`

如何理解: `int *p;`
↓ `*p`替代a
`int a[4];` → `int (*p)[4];`



6.3.3 二维数组与指针

注意：

1. 括号不能省略，不能写成`int *p[4];`
2. 中括号中的数字必须跟二维数组第二维大小相同

如： `int a[2][3];`

`int (*p)[4];`

`p=a;`



6.3.3 二维数组与指针

例6.14 利用行指针输出二维数组中的元素

```
#include<stdio.h>
```

```
int main( )
```

```
{
```

```
    int a[3][4]={1,3,5,7},{2,4,6,8},{9,10,11,12}};
```

```
    int (*p)[4], i, j;
```

使用数组名

```
    p=a;
```

```
    for( i=0; i<3; i++)
```

a[i][j]的值

```
        { for( j=0; j<4; j++)
```

```
            printf("%2d ", (*(p+i)+j) );
```

```
            printf("\n");
```

```
        }
```

```
    }
```



6.4 指针与字符串

- C语言中字符串是存放在字符数组中，并以\0作为结束标志。

【例】字符串的输出方式

```
#include <stdio.h>
```

H	e	l	l	o	\0				
---	---	---	---	---	----	--	--	--	--

```
int main( )
```

```
{   int i;   char str[10]="Hello"; //字符串初始化
```

```
    printf("方式1: ");
```

```
    printf("%s\n", str); //用格式字符s输出字符串
```

```
    printf("方式2: ");
```

```
    puts(str); //用字符串输出函数输出字符串
```

```
    printf("方式3: ");
```

```
    for(i=0; str[i]!='\0'; i++) //用循环逐个输出每个字符
```

```
        printf("%c", str[i]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```


6.4 指针与字符串

➤ 指向字符串的指针实际上就是指向一个字符数组的指针。

【例6.16】利用指针的不同方式，输出字符串。

```
#include <stdio.h>
```

```
int main( )
```

```
{  char str[10]="Hello", *p=str;
```

```
    int i;
```

```
    printf("方式1: ");
```

```
    printf("%s\n", p);
```

```
    printf("方式2: ");
```

```
    puts(p);
```

```
    printf("方式3: ");
```

```
    for(i=0; *(p+i)!='\0'; i++)
```

```
        printf("%c", *(p+i));
```

```
    printf("\n");
```

```
    printf("方式4: ");
```

```
    for(p=str; *p!='\0'; p++)
```

```
        printf("%c", *p);
```

```
    return 0;
```

```
}
```

定义字符指针变量p, 并进行初始化, 令p指向字符数组str

注意这两种输出方式的区别

6.4 指针与字符串

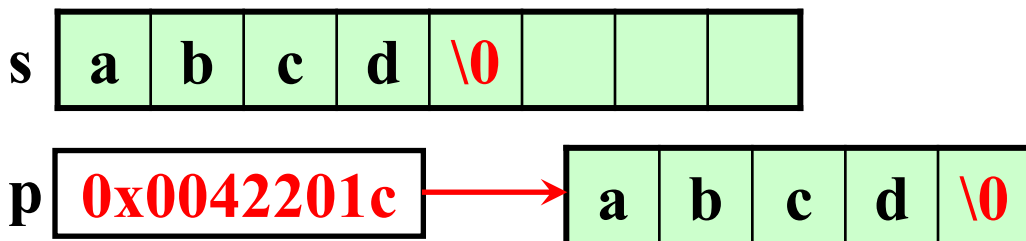
➤ 字符数组与字符指针变量的区别

1. 存储方式不同

字符数组——存放的是字符串中的字符和'\0'

字符指针变量——存放的是字符串的首地址

```
char s[8] = "abcd", *p = "abcd";
```



注意:

- C语言中**字符串常量**是按照字符数组来处理的。对程序中的字符串常量，系统会自动在内存中创建一个字符数组，将字符串的内容及'\0'保存在字符数组中。
- 虽然s和p都能代表字符串的首地址，但s是数组名，是一个常量，而p是一个指针变量，因此不能写 s++，但写 p++ 是对的。

6.4 指针与字符串

➤ 字符数组与字符指针变量的区别

2. 赋值方式不同

字符数组可以初始化,但不能赋值,赋值操作需借助`strcpy()`函数实现。

```
char  a[12], b[6] = "good";  
a[12]= "China";  
a= "China";
```

} 错 应该用: `strcpy(a, "China");`

字符指针既可以初始化,也可以赋值。

```
char *p1, *p2 = "abcd";  
p1 = "China" ;
```

// 可以将字符串常量直接赋给字符指针变量

6.4 指针与字符串

➤ 字符数组与字符指针变量的区别

3. 字符数组具有一片连续的存储空间，数组名即为该存储空间的首地址；字符指针变量的存储单元是用来存放字符串首地址的，但是在对字符指针变量赋值之前，该指针变量的值是不确定的，此时不应该对字符指针变量进行操作，否则可能出现异常。

```
char s1[5], s2[10];
```

```
scanf("%s", s1);
```

```
strcpy(s2, "Hello");
```

} 这种用法完全正确

```
char *p1, *p2;
```

```
scanf("%s", p1);
```

```
strcpy(p2, "Hello");
```

} 这种用法很危险，因为

6.4 指针与字符串

例 编写函数实现strcpy功能，即字符串复制

```
void copystr(char *from, char *to)
```

```
{
```

```
    while( *from != '\0' )
```

```
    {
```

```
        *to = *from ;
```

```
        from++;
```

```
        to++;
```

```
    }
```

```
    *to = '\0' ;
```

```
}
```

```
int main()
```

```
{
```

```
    char a[6] = "cat" , b[6] = "tiger";
```

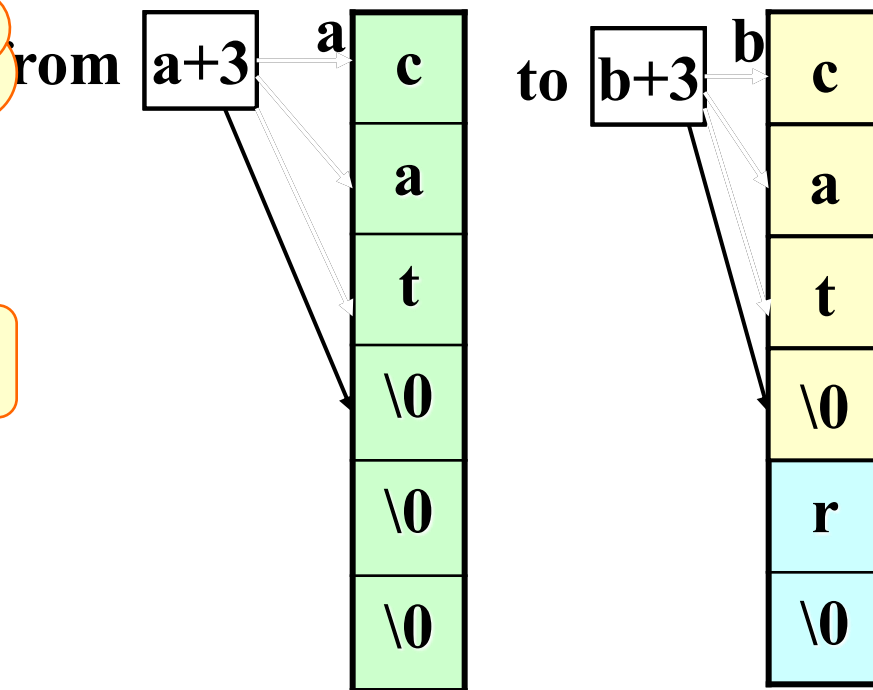
```
    copystr(a , b);
```

```
    puts(a);    puts(b);
```

```
}
```

可否
简化?

去掉会怎样?



6.4 指针与字符串

【例6.18】利用字符指针编写程序实现strcat的功能。

分析：

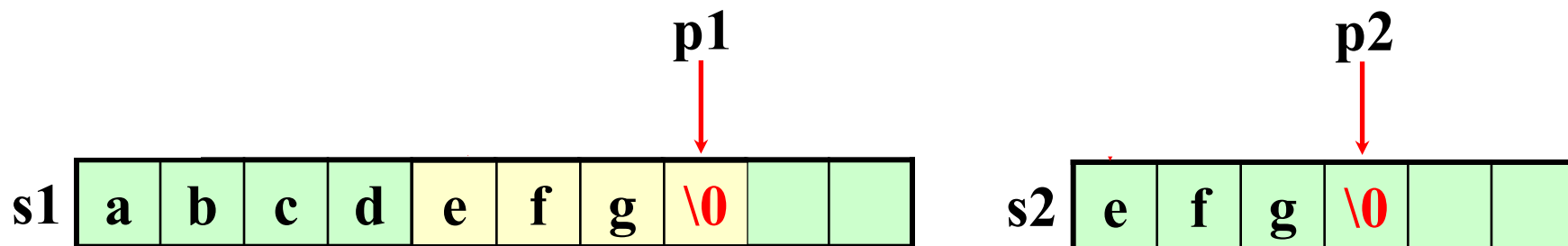
定义两个字符数组s1和s2，接受用户输入的字符串；

目标是实现将s2连接到s1的后面；

定义两个字符指针变量p1和p2，分别指向s1和s2；

想办法让p1指向字符串s1的末尾，即指向s1最后的结束符'\0'；

利用循环逐一将s2中的字符复制到s1的后面。



6.4 指针与字符串

【例6.18】 参考程序

```
#include <stdio.h>
```

```
int main( )
```

```
{ char s1[200], s2[100],*p1,*p2;
```

```
  p1=s1;    p2=s2;           // p1和p2分别指向s1和s2
```

```
  printf("Input first string: ");    gets(s1);
```

```
  printf("Input second string: ");    gets(s2);
```

```
  while(*p1!= '\0') //将p1移动到s1的末尾
```

```
    p1++;
```

```
  while((*p1=*p2)!= '\0') //将s2的内容接到s1后面
```

```
    { p1++; p2++; }
```

```
  printf("Catenated string: %s\n", s1); //输出连接后的内容
```

```
  return 0;
```

```
}
```

6.5 指针数组与多级指针

6.5.1 指针数组

- **概念：**一个**数组**的**元素**均为**指针类型**的数据，
则该数组称为**指针数组**

即：指针数组的每一个元素都是一个指针变量

定义格式： 类型名 *数组名[数组长度];

例如： int *a[4]; int float *b[20];

判断是否为指针数组：

int (*a)[5]; **不是** 是指向一维数组的指针。

int *(a[5]); **是** 等价于int *a[5]

注：一定要区分指针数组和指向一维数组的指针

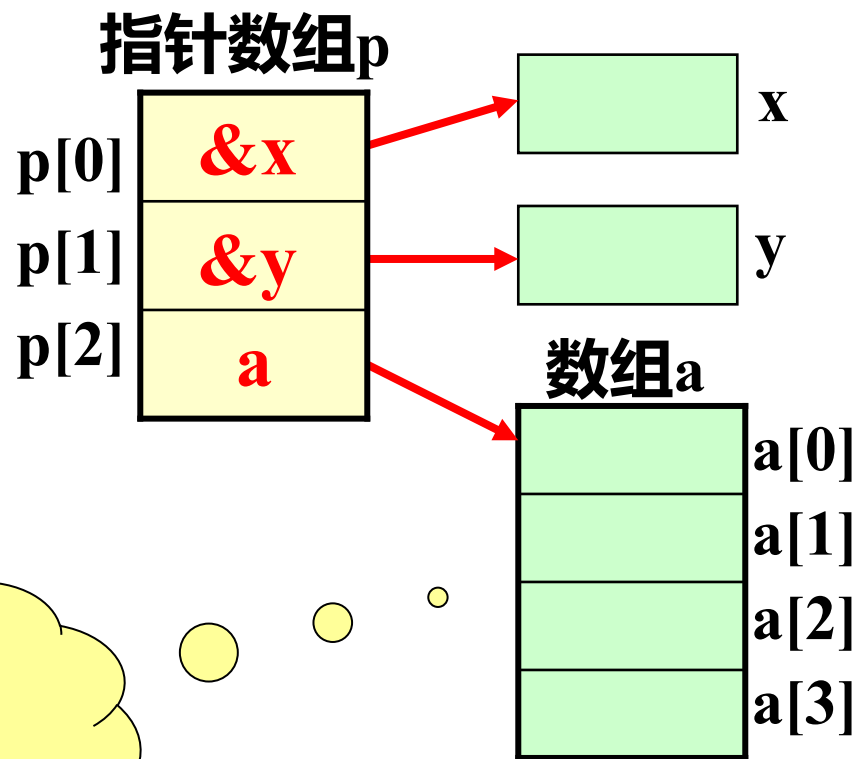


6.5.1 指针数组

由于指针数组的每个元素为指针，因此只能存储**地址**。

```
int *p[3];  
int x, y, a[4];  
p[0]=&x;  
p[1]=&y;  
p[2]=a;
```

p是一个指针数组, 它有3个元素, 每个元素是一个整型指针变量, 可指向一个整型数据



如果利用p访问数组a的a[1], 如何写?

可写成: $*(p[2]+1)$
或 $p[2][1]$

6.5.1 指针数组

例6.20: 对多个字符串进行排序

方法: 用二维字符数组编程实现

思考:

①二维字符数组定义成多大合适?

第一维的长度取决于有几个字符串
第二维的长度由最长的字符串决定

②字符串怎样比较和交换?

字符串的比较和交换应使用相应的
字符串函数

P	a	s	c	a	l	\0
V	C	\0				
B	a	s	i	c	\0	
J	a	v	a	\0		



排序

B	a	s	i	c	\0	
J	a	v	a	\0		
P	a	s	c	a	l	\0
V	C	\0				

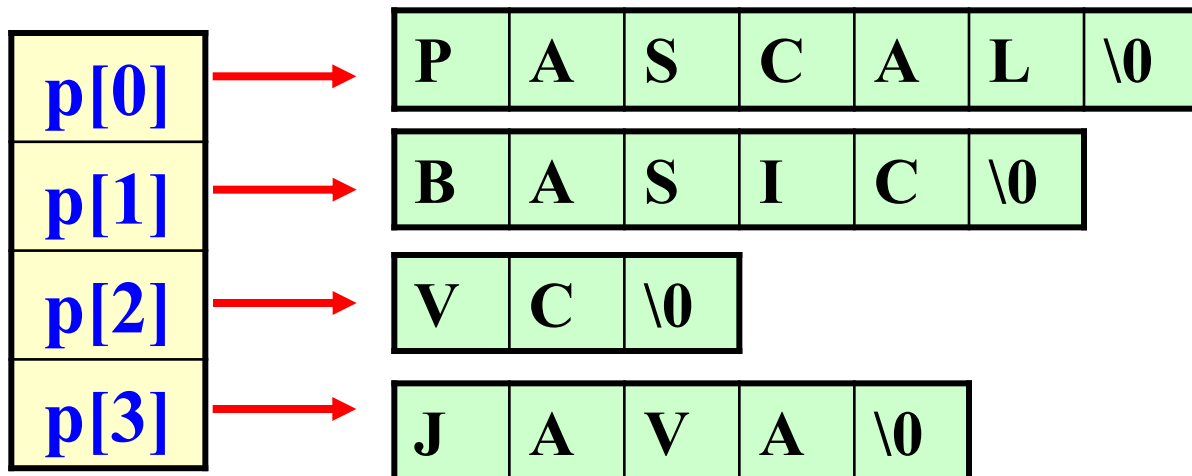
指针数组的优势-可以节省内存空间

如果使用二维数组存储长度不等的字符串，存在空间浪费

P	a	s	c	a	l	\0
V	C	\0				
B	a	s	i	c	\0	
J	a	v	a	\0		

闲置的内存空间

`char *p[4];`



使用字符指针数组可以指向多个**长度不等**的字符串，不存在空间浪费。

指针数组的优势 - 可以提高排序效率

使用二维数组排序，需要进行字符串位置交换

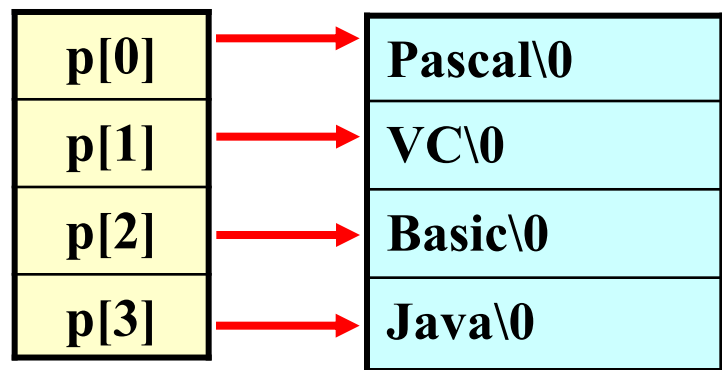
P	a	s	c	a	l	\0
V	C	\0				
B	a	s	i	c	\0	
J	a	v	a	\0		

排序后

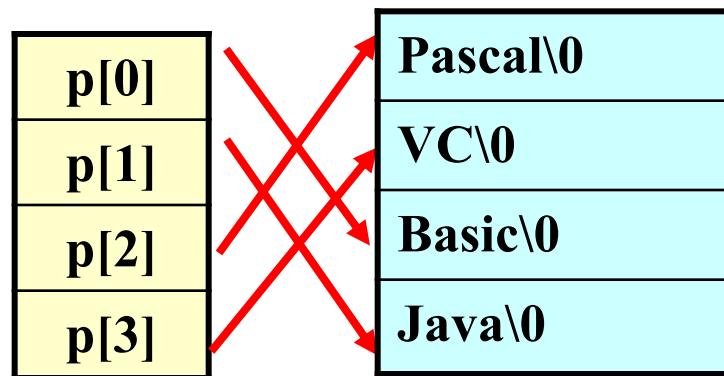


B	a	s	i	c	\0	
J	a	v	a	\0		
P	a	s	c	a	l	\0
V	C	\0				

使用指针数组排序，只需要交换指针变量的指向



排序后



6.5.1 指针数组

使用指针数组实现

```
#include<stdio.h>
#include<string.h>
#define N 4
void main( )
{ int i, j;   char *temp=NULL;
  char *p[N]={"Pascal", "VC", "Basic", "Java"};
  printf("\nBefore sorted:\n");
  for(i=0; i<N; i++)  puts(p[i]);
  for(i=0; i<N-1; i++)
    for(j=i+1; j<N; j++)
      if ( strcmp(p[i], p[j])>0)
        { temp=p[i];  p[i]=p[j];  p[j]=temp;  }
  printf("\nAfter sorted:\n");
  for(i=0; i<N; i++)  puts(p[i]);
}
```

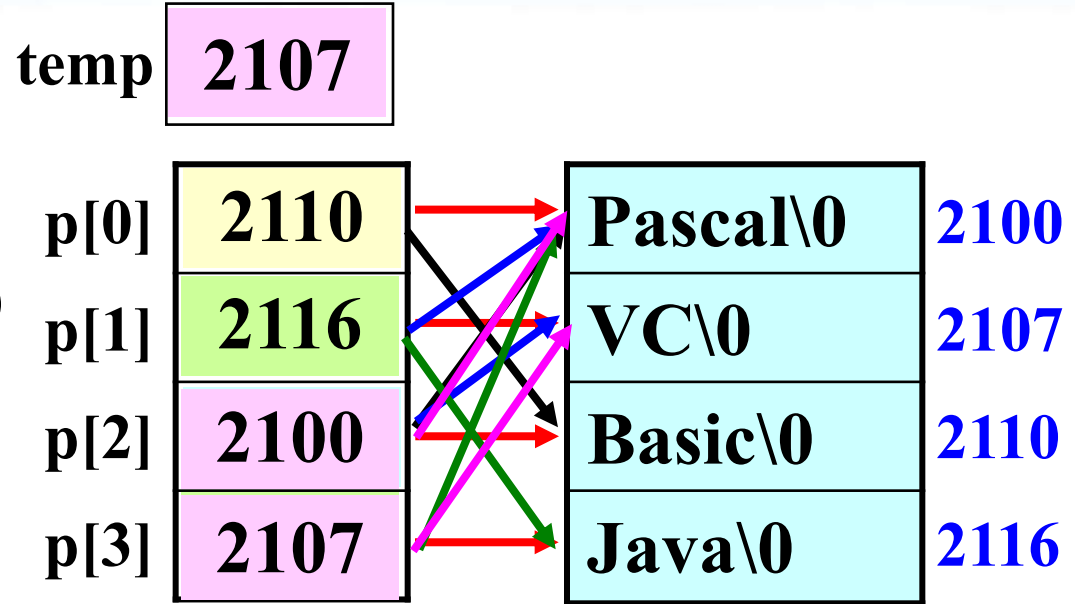
交换的是字符串的
起始地址



6.5.1 指针数组

排序过程演示

```
for(i=0; i<N-1; i++)  
    for(j=i+1; j<N; j++)  
        if ( strcmp(p[i], p[j])>0)  
            { temp=p[i];  
              p[i]=p[j];  
              p[j]=temp; }
```



i=0, j=1 i=0, j=2 i=0, j=3

P≠V

P>B

B≠J

temp=p[0];

p[0]=p[2];

p[2]=temp;

i=1, j=2

V>P

temp=p[1];

p[1]=p[2];

p[2]=temp;

i=1, j=3

P>J

temp=p[1];

p[1]=p[3];

p[3]=temp;

i=2, j=3

V>P

temp=p[2];

p[2]=p[3];

p[3]=temp;

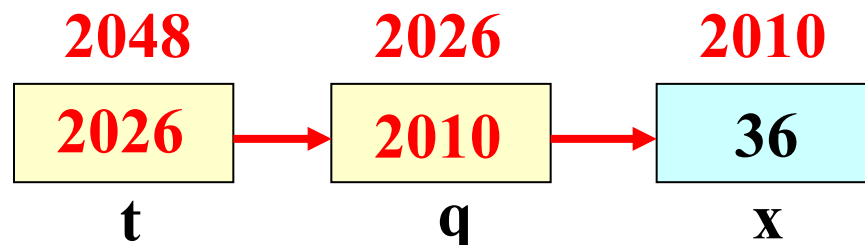
6.5.2 多级指针

```
int x, *q, **t;
```

```
x=36;
```

```
q=&x;
```

```
t=&q;
```



t即为指向指针的指针(也称二级指针)

思考:

q=45; **错,因为q是指针变量, 赋值号右侧必须是地址**

*q=45; **对**

*t=56; **错,因为t是二级指针变量, *t等价于q**

t=56; **对

6.6 指针与动态内存分配

➤ 动态内存分配(Dynamic Memory Allocation)

在程序运行时为程序分配内存的一种方法

◆ 什么时候需要使用动态内存分配?

例: 计算某班英语成绩的平均分,要求保存每个学生的成绩,
学生人数由键盘输入。

以前的方法是将数组定义的足够大!

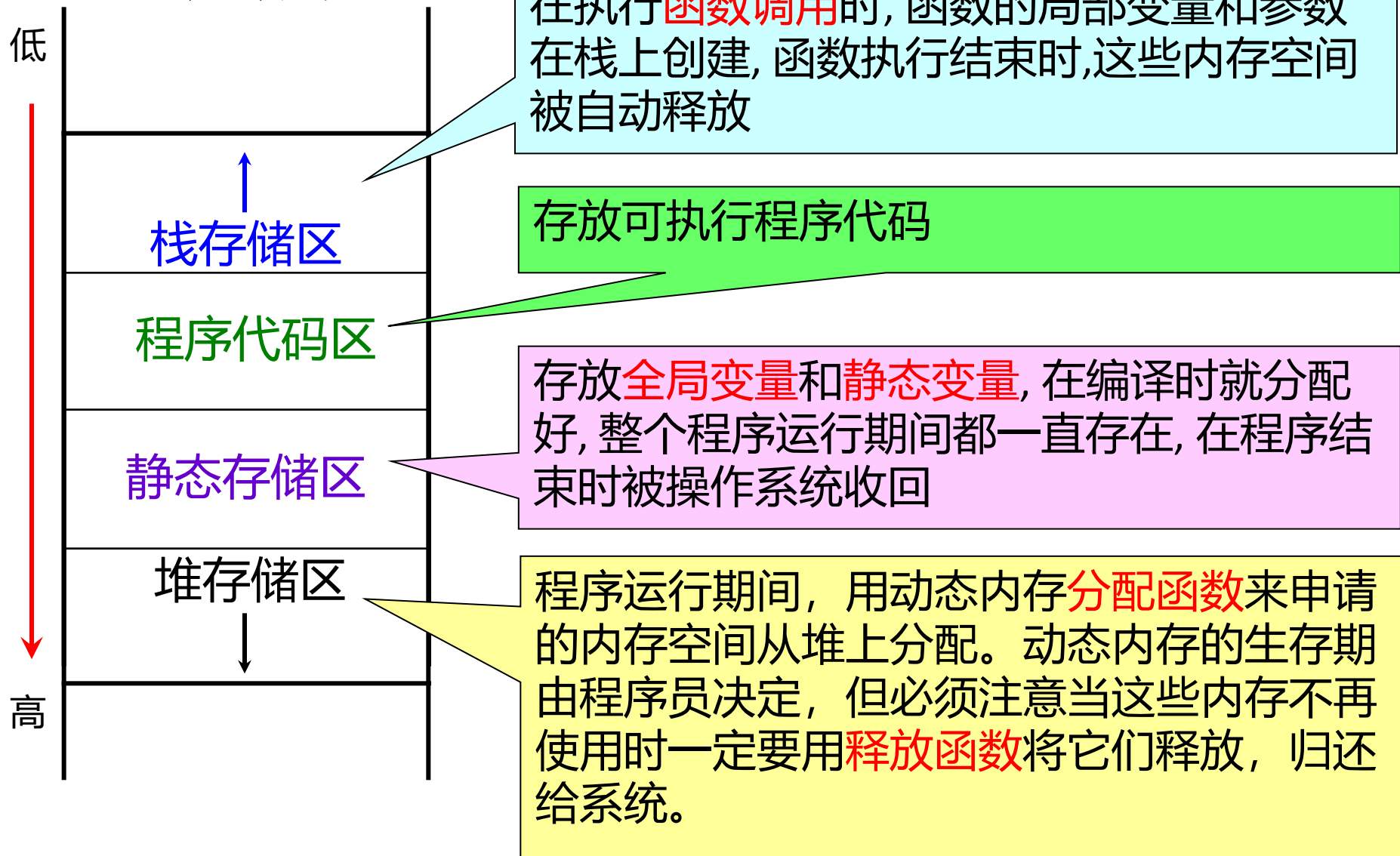
```
int a[60];  
int n, i;  
scanf("%d", &n);  
for(i=0; i<n; i++)  
    scanf("%d", &a[i]);
```

我们希望在输入n后,根据实际的人数定义一个动态数组,这样不会浪费存储空间

如果输入的n较小(15), 那么数组有多数以上的存储空间被浪费了

6.6 指针与动态内存分配

6.6.1 内存管理



6.6 指针与动态内存分配

6.6.2 void类型指针

void是一种特殊的类型，表示“无类型”。

注意： C语言中的变量不能定义为**void类型**，但允许定义**void类型的指针变量**，称为无类型指针变量，初始值通常设为**NULL**。

➤ void类型指针的使用

1. 一些标准函数的返回值是无类型指针，这种类型的指针不能直接赋给其他类型的指针变量，在使用时需要根据情况将无类型指针**强制转换**为其他类型的指针，
2. 函数的形式参数也可以定义为无类型指针，但在调用该函数时，不需要对实际参数进行强制转换。



6.6 指针与动态内存分配

6.6.2 动态内存分配和释放函数

使用动态内存分配函数必须使用**#include<stdlib.h>**文件包含命令

➤ 常用的动态内存分配函数有三个

1. malloc()函数

函数原型: **void * malloc(unsigned int size);**

函数功能: 在内存的动态存储区中分配一块长度为“size”字节的连续区域, 函数的返回值为该区域的首地址。如果系统不能提供足够的内存单元(即分配失败), 则函数将返回NULL。

调用malloc函数时, 必须用强制类型转换。

调用形式: **(类型标识符 *) malloc (size)**



6.6.2 动态内存分配和释放函数

`int *p=NULL;` //定义指针变量，通常将其初始化为空指针

`double *q=NULL;`

`p= (int *) malloc (4);`

申请一个长度为4个字节的存储空间，并将其首地址赋给int型的指针变量，则该空间可存放int型数据

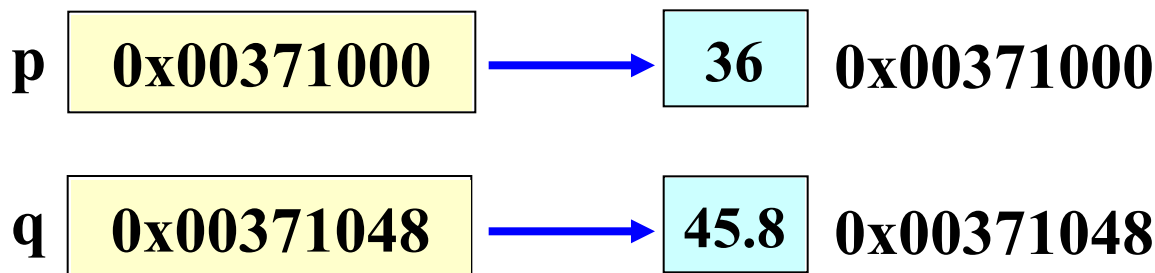
`q= (double *) malloc ( sizeof(double) );`

`*p=36;`

`*q=45.8;`

强制类型转换

如果不知道某种类型数据所占内存空间的字节数，可用sizeof()进行计算



6.6.2 动态内存分配和释放函数

2. calloc()函数

函数原型:

void * calloc (unsigned int n, unsigned int size);

函数功能: 在内存动态存储区中分配 **n** 块长度为 “size” 字节的连续区域, 函数的返回值为该区域的首地址。若分配失败返回 NULL。

说明:

参数**n**表示向系统申请的存储空间的数量

参数**size**表示每个存储空间所占的字节数



6.6.2 动态内存分配和释放函数

2. calloc()函数

例: `float *p=NULL;`
`p=(float *) calloc (10, sizeof(float));`

申请10个连续的float型的存储单元, 并将其首地址赋给float型的指针变量, 该存储空间总的字节数为 $10 \times \text{sizeof(float)}$

显然, 用calloc()函数开辟的存储空间相当于一个一维数组, 第1个参数决定了一维数组的大小, 第2个参数决定了数组元素的类型, 函数的返回值就是数组的首地址。

6.6.2 动态内存分配和释放函数

3. free()函数

函数原型：**void free(void *ptr);**

函数功能：释放ptr所指向的一块内存空间，ptr是一个void类型的指针变量，它指向被释放区域的首地址。

说明：

- ① 参数ptr的值不是任意的地址，**必须是执行malloc或calloc函数所返回的地址。**
- ② ptr是**void类型指针变量**，但调用free函数时不需要进行强制类型转换，计算机系统会自动转换。
- ③ 该函数执行后，将指针ptr所指向的存储空间交还给系统，系统可再重新分配。

6.6.2 动态内存分配和释放函数

3. free()函数

例：

```
int i, *p=NULL;  
p=(int *) calloc (3, sizeof(int) );  
for(i=0; i<3; i++)  
    *(p+i)=i+1;  
free(p);
```

p

0x0012ff70

注意： free函数释放的是p所指向的动态存储空间, 变量p并没有被释放

6.6.2 动态内存分配和释放函数

【例6.23】 计算某班英语成绩平均分(用一维动态数组实现)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main( )
```

```
{ int n, i, *p=NULL; double sum=0, ave;
```

```
scanf("%d", &n);
```

p=(int *) malloc (n*4);

```
p=(int *) calloc ( n, sizeof(int));
```

```
if (p==NULL)
```

```
{ printf("\n No enough memory!\n"); exit(1); }
```

```
for( i=0; i<n; i++)
```

```
{ scanf("%d", p+i);
```

```
sum=sum+*(p+i);
```

```
}
```

堆是有限的, 分配内存后必须检查指针变量是否为空(使用空指针可能导致程序瘫痪)

```
ave=sum/n;
```

注意指针变量的使用方法

```
printf("\n ave=%.2lf \n", ave);
```

```
free(p);
```

```
return 0;
```

```
}
```



**培养创新思维
提升编程能力**